DÉMO 004-A · CHANGEMENTS ENTRE LES ÉTAPES

Custom hooks

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-toggle-btn (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-toggle-checkbox (3 fichiers)
3. **Étape 2 → 3** · etape-3-network-status (4 fichiers)
4. **Étape 3 → 4** · etape-4-users (3 fichiers)

Étape 1 etape-1-toggle-btn : point de départ

Un toggle ON / OFF avec un bouton (useState)

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (104 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

Présents mais pas encore branchés (non affichés) : src/components/BadExample.jsx, src/components/Comments.jsx, src/components/Layout.jsx, src/components/ToggleCheckbox.jsx, src/components/Users.jsx, src/hooks/useFetchApi.ts, src/hooks/useToogle.ts, src/hooks/usetNetworkStatus.ts

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 14 lignes

```jsx
import "./App.css";
import ToggleBtn from "./components/ToggleBtn";
import ToggleCheckbox from "./components/ToggleCheckbox";
import Layout from "./components/Layout";
import Users from "./components/Users";
import Comments from "./components/Comments";

import BadExample from "./components/BadExample";

function App() {
  return <ToggleBtn />;
}

export default App;
```

DÉPART **src/components/ToggleBtn.jsx** 23 lignes

```jsx
import { useState } from "react";

function ToggleBtn() {
  const [value, setValue] = useState(false);
```

```
5
6     const setToggle = (value) => {
7       setValue(!value);
8     };
9
10    return (
11      <div>
12        <h2 className="container">Toggle ON / OFF</h2>
13        <button
14          className={value ? "button button--on" : "button button--off"}
15          onClick={() => setToggle(value)}
16        >
17          {value ? "ON" : "OFF"}
18        </button>
19      </div>
20    );
21  }
22
23  export default ToggleBtn;
```

Étape 1 → 2 etape-1-toggle-btn → etape-2-toggle-checkbox

Le même toggle avec une checkbox (le hook useToggle est écrit dans src/hooks/useToogle.ts, mais pas encore branché : ToggleCheckbox garde son useState)

COMPARÉ `src/components/ToggleBtn.jsx → src/components/ToggleCheckbox.jsx` +10 −8

```
@@ -1,6 +1,6 @@
  import { useState } from "react";

- function ToggleBtn() {
+ function ToggleSwitch() {
    const [value, setValue] = useState(false);

    const setToggle = (value) => {
@@ -10,14 +10,16 @@
    return (
      <div>
        <h2 className="container">Toggle ON / OFF</h2>
-       <button
-         className={value ? "button button--on" : "button button--off"}
-         onClick={() => setToggle(value)}
-       >
-         {value ? "ON" : "OFF"}
-       </button>
+       <label className="switch">
+         <input
+           type="checkbox"
+           checked={value}
+           onChange={(e) => setValue(e.target.checked)}
+         />
+         <span>{value ? "ON" : "OFF"}</span>
+       </label>
      </div>
    );
  }

- export default ToggleBtn;
+ export default ToggleSwitch;
```

MODIFIÉ `src/App.jsx` +1 −1

```
@@ -8,7 +8,7 @@
  import BadExample from "./components/BadExample";

  function App() {
-   return <ToggleBtn />;
+   return <ToggleCheckbox />;
  }

  export default App;
```

UTILISÉ `src/components/ToggleCheckbox.jsx` +25 −0

```
@@ -1,0 +1,25 @@
+ import { useState } from "react";
+
+ function ToggleSwitch() {
+   const [value, setValue] = useState(false);
+
+   const setToggle = (value) => {
+     setValue(!value);
+   };
```

```
+
+   return (
+     <div>
+       <h2 className="container">Toggle ON / OFF</h2>
+       <label className="switch">
+         <input
+           type="checkbox"
+           checked={value}
+           onChange={(e) => setValue(e.target.checked)}
+         />
+         <span>{value ? "ON" : "OFF"}</span>
+       </label>
+     </div>
+   );
+ }
+
+ export default ToggleSwitch;
```

Étape 2 → 3 etape-2-toggle-checkbox → etape-3-network-status

Le hook useNetworkStatus : listeners online / offline dans useEffect

COMPARÉ `src/components/ToggleCheckbox.jsx → src/components/Layout.jsx` +17 −19

```
@@ -1,25 +1,23 @@
- import { useState } from "react";
+ import useNetworkStatus from "../hooks/usetNetworkStatus";

- function ToggleSwitch() {
-   const [value, setValue] = useState(false);
-
-   const setToggle = (value) => {
-     setValue(!value);
-   };
+ function Layout() {
+   const { isOnline } = useNetworkStatus();

    return (
-     <div>
-       <h2 className="container">Toggle ON / OFF</h2>
-       <label className="switch">
-         <input
-           type="checkbox"
-           checked={value}
-           onChange={(e) => setValue(e.target.checked)}
-         />
-         <span>{value ? "ON" : "OFF"}</span>
-       </label>
-     </div>
+     <>
+       {!isOnline && (
+         <div style={{ padding: 10, background: "#ffdddd", color: "#900" }}>
+           Vous êtes hors ligne
+         </div>
+       )}
+
+       {isOnline && (
+         <div style={{ padding: 10, background: "#ddffdd", color: "#090" }}>
+           Vous êtes connecté
+         </div>
+       )}
+     </>
    );
  }

- export default ToggleSwitch;
+ export default Layout;
```

MODIFIÉ `src/App.jsx` +1 −1

```
@@ -8,7 +8,7 @@
  import BadExample from "./components/BadExample";

  function App() {
-   return <ToggleCheckbox />;
+   return <Layout />;
  }

  export default App;
```

UTILISÉ `src/components/Layout.jsx` +23 −0

```
@@ -1,0 +1,23 @@
+ import useNetworkStatus from "../hooks/usetNetworkStatus";
+
+ function Layout() {
+   const { isOnline } = useNetworkStatus();
+
+   return (
+     <>
+       {!isOnline && (
+         <div style={{ padding: 10, background: "#ffdddd", color: "#900" }}>
+           Vous êtes hors ligne
+         </div>
+       )}
+
+       {isOnline && (
+         <div style={{ padding: 10, background: "#ddffdd", color: "#090" }}>
+           Vous êtes connecté
+         </div>
+       )}
+     </>
+   );
+ }
+
+ export default Layout;
```

UTILISÉ **src/hooks/usetNetworkStatus.ts** +22 −0

```
@@ -1,0 +1,22 @@
+ import { useEffect, useState } from "react";
+
+ function useNetworkStatus() {
+   const [isOnline, setIsOnline] = useState(() => navigator.onLine);
+
+   useEffect(() => {
+     const onOnline = () => setIsOnline(true);
+     const onOffline = () => setIsOnline(false);
+
+     window.addEventListener("online", onOnline);
+     window.addEventListener("offline", onOffline);
+
+     // Nettoyage : on retire les écouteurs quand le composant disparaît
+     return () => {
+       window.removeEventListener("online", onOnline);
+       window.removeEventListener("offline", onOffline);
+     };
+   }, []);
+
+   return { isOnline };
+ }
+ export default useNetworkStatus;
```

Étape 3 → 4 etape-3-network-status → etape-4-users

La page Users : fetch avec loading / error (point de départ de l'exercice 11)

COMPARÉ **src/components/Layout.jsx → src/components/Users.jsx** +52 −16

```
@@ -1,23 +1,59 @@
- import useNetworkStatus from "../hooks/usetNetworkStatus";
+ import { useEffect, useState } from "react";

- function Layout() {
-   const { isOnline } = useNetworkStatus();
+ function Users() {
+   const [users, setUsers] = useState([]);
+   const [isLoading, setIsLoading] = useState(false);
+   const [isError, setIsError] = useState(false);
+
+   const getData = async () => {
+     setIsLoading(true);
+     try {
+       const response = await fetch(
+         "https://jsonplaceholder.typicode.com/users"
+       );
+       const json = await response.json();
+       setUsers(json);
+     } catch (error) {
+       setIsError(true);
+     } finally {
+       setIsLoading(false);
+     }
+   };
+
+   // useEffect avec [] : getData tourne une seule fois, au montage
+   useEffect(() => {
+     getData();
+   }, []);

    return (
-     <>
-       {!isOnline && (
-         <div style={{ padding: 10, background: "#ffdddd", color: "#900" }}>
-           Vous êtes hors ligne
-         </div>
+     <div>
+       <h2 className="container">Welcome to users page</h2>
+       {isLoading ? (
+         <div className="loading">Loading ...</div>
+       ) : (
+         <table className="table">
+           <thead>
+             <tr>
+               <th className="header">Id</th>
+               <th className="header">Name</th>
+               <th className="header">Email</th>
+             </tr>
+           </thead>
+           <tbody>
+             {users.map((user) => {
+               return (
+                 <tr key={user.id}>
+                   <td className="item">{user.id}</td>
+                   <td className="item">{user.name}</td>
+                   <td className="item">{user.email}</td>
+                 </tr>
```

```
                50 +                );
                51 +              })}
                52 +            </tbody>
                53 +          </table>
 12             54           )}
 13             -
 14             -       {isOnline && (
 15             -         <div style={{ padding: 10, background: "#ddffdd", color: "#090" }}>
 16             -           Vous êtes connecté
 17             -         </div>
 18             -       )}
 19             -     </>
                55 +     </div>
 20             56     );
 21             57   }
 22             58
 23             - export default Layout;
                59 + export default Users;
```

MODIFIÉ **src/App.jsx** +1 −1

```
@@ -8,7 +8,7 @@
  8  8   import BadExample from "./components/BadExample";
  9  9
 10 10   function App() {
 11     -   return <Layout />;
        11 +   return <Users />;
 12 12   }
 13 13
 14 14   export default App;
```

UTILISÉ **src/components/Users.jsx** +59 −0

```
@@ -1,0 +1,59 @@
  1 + import { useEffect, useState } from "react";
  2 +
  3 + function Users() {
  4 +   const [users, setUsers] = useState([]);
  5 +   const [isLoading, setIsLoading] = useState(false);
  6 +   const [isError, setIsError] = useState(false);
  7 +
  8 +   const getData = async () => {
  9 +     setIsLoading(true);
 10 +     try {
 11 +       const response = await fetch(
 12 +         "https://jsonplaceholder.typicode.com/users"
 13 +       );
 14 +       const json = await response.json();
 15 +       setUsers(json);
 16 +     } catch (error) {
 17 +       setIsError(true);
 18 +     } finally {
 19 +       setIsLoading(false);
 20 +     }
 21 +   };
 22 +
 23 +   // useEffect avec [] : getData tourne une seule fois, au montage
 24 +   useEffect(() => {
 25 +     getData();
 26 +   }, []);
 27 +
 28 +   return (
 29 +     <div>
 30 +       <h2 className="container">Welcome to users page</h2>
 31 +       {isLoading ? (
```

```
+         <div className="loading">Loading ...</div>
+       ) : (
+         <table className="table">
+           <thead>
+             <tr>
+               <th className="header">Id</th>
+               <th className="header">Name</th>
+               <th className="header">Email</th>
+             </tr>
+           </thead>
+           <tbody>
+             {users.map((user) => {
+               return (
+                 <tr key={user.id}>
+                   <td className="item">{user.id}</td>
+                   <td className="item">{user.name}</td>
+                   <td className="item">{user.email}</td>
+                 </tr>
+               );
+             })}
+           </tbody>
+         </table>
+       )}
+     </div>
+   );
+ }
+
+ export default Users;
```